ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)»

Отчет по лабораторной работе №3 по предмету «Операционные системы и архитектура компьютера»

Институт №8 «Компьютерные науки и прикладная математика»

Кафедра ЦОП ВО «ТОП-ИТ» Учебная группа М8О-205БВ-25

ФИО обучающегося Изотов Арсений Антонович

29 октября

2026г.

дата

Москва

2026

ВВЕДЕНИЕ

Цель работы: приобретение практических навыков управления потоками в ОС и обеспечения синхронизации между потоками.

Задача: составить программу на языке C, вычисляющую детерминант матрицы через определение (разложение по первой строке) в многопоточном режиме. Максимальное количество одновременно работающих потоков задаётся ключом запуска -t N. Необходимо продемонстрировать количество потоков средствами ОС и исследовать зависимость ускорения и эффективности от входных данных и числа потоков.

Вариант: 9.

ОС: Windows. API**:** Win32. Среда сборки: CMake + MinGW.

АРХИТЕКТУРА РЕШЕНИЯ

Архитектура программы и схема параллелизма потоков представлены на таблице 1 и рисунке 1, соответственно.

Файл	Назначение
src/main.c	Разбор аргументов (-t, -n, -s, -p), создание семафора/мьютекса, замер времени, вывод статистики
src/det.c/.h	det_sequential (базовая линия), worker (функция потока), det_parallel (запуск потоков, WaitForMultipleObjects, сбор результатов
src/common.c / src/common.h	Утилита печати системных ошибок WinAPI через FormatMessage
src/matrix.c/.h	Тип Matrix, выделение/освобождение, заполнение, минор

Таблица 1 – архитектура программы

```
det_parallel(m, ctx)
  ├── for j in 0..n-1:
  │     CreateThread(worker, &tasks[j])   ← n потоков, по одному на M_{0j}
  │
  ├── worker(Task *t):
  │     WaitForSingleObject(sem)          ← блокируется, если max_threads слотов заняты
  │     stats_enter(ctx)                  ← active++ под мьютексом, обновление peak
  │     M = matrix_minor(m, 0, j)
  │     t->result = sign(j) * a[0][j] * det_sequential(M)
  │     stats_leave(ctx)                  ← active-- под мьютексом
  │     ReleaseSemaphore(sem, 1, NULL)    ← освобождает слот
  │
  └── WaitForMultipleObjects(n, threads) ← ждём все (батчами по 64)
      sum += tasks[j].result
```

Рисунок 1 – схема параллелизма

КЛЮЧЕВЫЕ МОМЕНТЫ КОДА

1) Параллельный запуск с ограничением семафором. Код представлен на рисунках 2–7.

```
static DWORD WINAPI worker(LPVOID arg) {
    Task *t = (Task *)arg;
    SyncCtx *c = t->ctx;

    // ограничитель: одновременно не более max_threads
    if (WaitForSingleObject( hHandle: c->sem,  dwMilliseconds: INFINITE) != WAIT_OBJECT_0) {
        t->error = 1;
        return 0;
    }

    LONG now = stats_enter(c);
    printf( format: "  [thread TID=%lu] j=%d started (active=%ld)\n",
           GetCurrentThreadId(), t->j, now);
```

Рисунок 2 – параллельный запуск (worker, начало)

```
    Matrix *sub = matrix_minor(t->m,  row: 0,  col: t->j);
    if (!sub) {
        t->error = 1;
    } else {
        int sub_det = det_sequential( m: sub);
        matrix_free( m: sub);
        long long term = (long long)llround(t->m->a[t->j]) * sub_det;
        t->result = (t->j % 2 == 0) ? term : -term;
    }

    stats_leave(c);
    ReleaseSemaphore( hSemaphore: c->sem,  lReleaseCount: 1,  lpPreviousCount: NULL);
    return 0;
}
```

Рисунок 3 – параллельный запуск (worker, конец)

```
int det_parallel(const Matrix *m, SyncCtx *ctx) {
    if (m->n == 1) return (int)llround(m->a[0]);
    if (m->n == 2)
        return (int)llround(m->a[0] * m->a[3] - m->a[1] * m->a[2]);

    int n = m->n;
    Task   *tasks   = (Task   *)calloc( NumOfElements: (size_t)n,  SizeOfElements: sizeof(*tasks));
    HANDLE *threads = (HANDLE *)calloc( NumOfElements: (size_t)n,  SizeOfElements: sizeof(*threads));
    if (!tasks || !threads) {
        perror( ErrMsg: "calloc");
        free( Memory: tasks); free( Memory: threads);
        exit( Code: 1);
    }
```

Рисунок 4 – параллельный запуск (det_parallel, начало)

```
int created = 0;
for (int j = 0; j < n; ++j) {
    tasks[j] = (Task){ .m = m, .j = j, .result = 0, .ctx = ctx, .error = 0 };
    threads[j] = CreateThread( lpThreadAttributes: NULL, dwStackSize: 0, lpStartAddress: worker,
                               lpParameter: &tasks[j], dwCreationFlags: 0, lpThreadId: NULL);
    if (!threads[j]) {
        print_error( prefix: "CreateThread");
        for (int k = 0; k < created; ++k) {
            WaitForSingleObject( hHandle: threads[k], dwMilliseconds: INFINITE);
            CloseHandle( hObject: threads[k]);
        }
        free( Memory: tasks); free( Memory: threads);
        exit( Code: 1);
    }
    ++created;
    InterlockedIncrement( Addend: &ctx->total_spawned);
}
```

Рисунок 5 – параллельный запуск (det_parallel, продолжение)

```
    DWORD remaining = (DWORD)n, offset = 0;
    while (remaining > 0) {
        DWORD batch = remaining > MAXIMUM_WAIT_OBJECTS
                    ? MAXIMUM_WAIT_OBJECTS : remaining;
        DWORD w = WaitForMultipleObjects( nCount: batch, lpHandles: &threads[offset],
                                          bWaitAll: TRUE, dwMilliseconds: INFINITE);
        if (w == WAIT_FAILED) {
            print_error( prefix: "WaitForMultipleObjects");
            for (DWORD k = 0; k < n; ++k) CloseHandle( hObject: threads[k]);
            free( Memory: tasks); free( Memory: threads);
            exit( Code: 1);
        }
        offset    += batch;
        remaining -= batch;
    }
```

Рисунок 6 – параллельный запуск (det_parallel, еще продолжение)

```
    long long sum = 0;
    int had_error = 0;
    for (int j = 0; j < n; ++j) {
        if (tasks[j].error) had_error = 1;
        sum += tasks[j].result;
        CloseHandle( hObject: threads[j]);
    }
    free( Memory: tasks);
    free( Memory: threads);

    if (had_error) {
        fprintf( stream: stderr,  format: "one or more workers failed\n");
        exit( Code: 1);
    }
    return (int)sum;
}
```

Рисунок 7 – параллельный запуск (det_parallel, конец)

2)Синхронизация счетчиков. Код на рисунке 8.

```
static LONG stats_enter(SyncCtx *c) {
    WaitForSingleObject( hHandle: c->mtx_stats,  dwMilliseconds: INFINITE);
    LONG now = InterlockedIncrement( Addend: &c->active);
    if (now > c->peak_active) c->peak_active = now;
    ReleaseMutex( hMutex: c->mtx_stats);
    return now;
}

static void stats_leave(SyncCtx *c) {
    WaitForSingleObject( hHandle: c->mtx_stats,  dwMilliseconds: INFINITE);
    InterlockedDecrement( Addend: &c->active);
    ReleaseMutex( hMutex: c->mtx_stats);
}
```

Рисунок 8 – синхронизация счетчиков

3) Создание семафора и мьютекса представлено на рисунке 9.

```
SyncCtx ctx = {0};
ctx.sem       = CreateSemaphoreA( lpSemaphoreAttributes: NULL,  lInitialCount: max_threads,
                                  lMaximumCount: max_threads,  lpName: NULL);
ctx.mtx_stats = CreateMutexA( lpMutexAttributes: NULL,  bInitialOwner: FALSE,  lpName: NULL);
if (!ctx.sem || !ctx.mtx_stats) {
    print_error( prefix: "CreateSemaphore/Mutex");
    if (ctx.sem)       CloseHandle( hObject: ctx.sem);
    if (ctx.mtx_stats) CloseHandle( hObject: ctx.mtx_stats);
    matrix_free(m);
    return 1;
}
```

Рисунок 9 – создание семафора и мьютекса

РЕЗУЛЬТАТЫ РАБОТЫ

1) Успешный запуск с ограничением -t 4 представлен на рисунке 10.

```
Process PID = 17508
Matrix size = 4, max_threads = 4, seed = 42

  [thread TID=29552] j=0 started (active=1)
  [thread TID=1444] j=1 started (active=2)
  [thread TID=3064] j=2 started (active=3)
  [thread TID=1152] j=3 started (active=4)

det (sequential) = 2202
det (parallel)   = 2202
match: YES

Timing:
  sequential: 0.000006 s
  parallel:   0.009704 s
  speedup:    0.00x

Threads:
  max_threads (key): 4
  total_spawned:     4
  peak_active:       4
```

Рисунок 10 – запуск с 4 потоками

2) Исследование зависимости ускорения и эффективности от количества потоков. При сравнении -t 1, -t 4 и -t 8 при -n 10 получаем:

   -t 1: sequential=0.369597s, parallel=0.396048s, speedup=0.93x, peak_active=1

   -t 4: sequential=0.388402s, parallel=0.130195s, speedup=2.98x, peak_active=4

   -t 8: sequential=0.395551s, parallel=0.096813s, speedup=4.09x, peak_active=8

Для исследования масштабируемости параллельной версии был зафиксирован размер матрицы n = 10 и выполнены три запуска с разным ограничением числа одновременно работающих потоков: -t 1, -t 4 и -t 8. В каждом запуске замерялось время работы параллельной версии, а также фиксировались счётчики total_spawned (сколько всего потоков создано) и peak_active (пик одновременно активных потоков). За базовую линию принято время параллельной версии при t = 1 - это честное сравнение «та же программа, но с ограничением в один поток», без влияния накладных расходов на создание потоков. Ускорение считалось как $S(t) = T(1)/T(t)$, эффективность - как $E(t) = S(t)/t$.

При t = 1 время работы параллельной версии составило 0.396 с, при t = 4 - 0.130 с, при t = 8 - 0.097 с. Соответственно ускорение: 1.00× при t = 1, 3.04× при t = 4 и 4.09× при t = 8. Эффективность: 100%, 76% и 51%. Счётчик peak_active во всех трёх запусках точно совпал с ключом -t (1, 4 и 8 соответственно), что подтверждает корректную работу счётного семафора: одновременно в рабочей секции никогда не находилось больше потоков, чем разрешено ключом. При этом total_spawned во всех запусках был равен 10 — то есть создаётся по одному потоку на каждый минор, а семафор ограничивает не количество созданных потоков, а количество одновременно работающих.

Ускорение растёт с увеличением числа потоков, но не линейно. При переходе от 1 к 4 потокам ускорение почти линейное (3.04× на 4 потоках, эффективность 76%) - 10 миноров первого уровня независимы, и основное время работы уходит именно на них, поэтому параллелизм хорошо окупается. При переходе к 8 потокам ускорение растёт до 4.09×, но эффективность падает до 51%: дальнейший рост числа потоков даёт всё меньший прирост. Причина - закон Амдала. В программе есть последовательные части, которые не параллелятся: создание 10 потоков через CreateThread (это ~300 мкс суммарно), копирование элементов в matrix_minor, выделение и освобождение памяти через calloc/free, суммирование результатов в родителе. Их доля составляет примерно 12% работы, что и ограничивает ускорение сверху. Кроме того, число независимых задач на верхнем уровне рекурсии равно n = 10 - это естественный предел параллелизма для данного алгоритма. При t = 8

два потока вынуждены ждать на семафоре, пока освободятся слоты, и часть времени простаивает.

Отдельно стоит отметить результат при t = 1: параллельная версия оказалась медленнее последовательной (0.396 с против 0.370 с, ускорение относительно последовательной всего 0.93×). Это не ошибка, а важная демонстрация того, что параллелизм не бесплатен. При t = 1 все 10 потоков создаются, но проходят через семафор по одному: полезная работа та же, что у последовательной версии, но добавляются накладные расходы на 10 вызовов CreateThread, 10 пар Wait/Release на семафоре, 10 пар Wait/Release на мьютексе и переключения контекста между потоками. Суммарно это даёт около 26 мс лишнего времени. Отсюда следует практический вывод: параллелизм окупается только при достаточном объёме работы на поток; при малом объёме он вредит.

ВЫВОДЫ

В ходе лабораторной работы:

1. Освоены WinAPI-примитивы управления потоками: CreateThread, WaitForMultipleObjects, WaitForSingleObject, GetCurrentThreadId.
2. Реализована синхронизация через счётный семафор (CreateSemaphoreA) для ограничения параллелизма ключом -t и мьютекс (CreateMutexA) для защиты общих счётчиков active / peak_active от гонки данных.
3. Отработаны инженерные требования: проверка кодов возврата каждого системного вызова, освобождение всех хендлов на всех ветках исполнения, отсутствие deadlock (семафор не удерживается дольше нужного).
4. Проведено исследование зависимости ускорения и эффективности от n и t. Показано, что:
    - при малых n параллелизм вредит из-за накладных расходов;
    - при средних n ускорение ≈ 2–4×;
    - пик активных потоков всегда равен -t - семафор работает корректно.

Вывод: WinAPI-примитивы предоставляют полноценную замену POSIX-связке. Корректная синхронизация через семафор ограничивает параллелизм, а мьютекс защищает общие данные от гонки данных. Параллелизм эффективен только при достаточном объёме работы на поток.